

PROJETO E IMPLEMENTAÇÃO DE BANCO DE DADOS

Etapa 2 – Trabalho

Grupo 11 - Integrantes:

- Anna Carolina Brito Santos Farias - 811448
- Maria Luiza Fernandes Prestes Cesar - 832374
- Nathália Brasilino Gimenes - 812076
- Yasmin Victoria Oliveira - 812308

1. TEMA

Gestão Acadêmica: Alunos, Matrículas, Turmas, Notas, Frequência, Cursos.

2. ESPECIFICAÇÃO

Em um banco de dados de um sistema de Gestão Acadêmica é necessário guardar informações sobre **alunos, cursos, grades curriculares, disciplinas, professores, turmas e matrículas**.

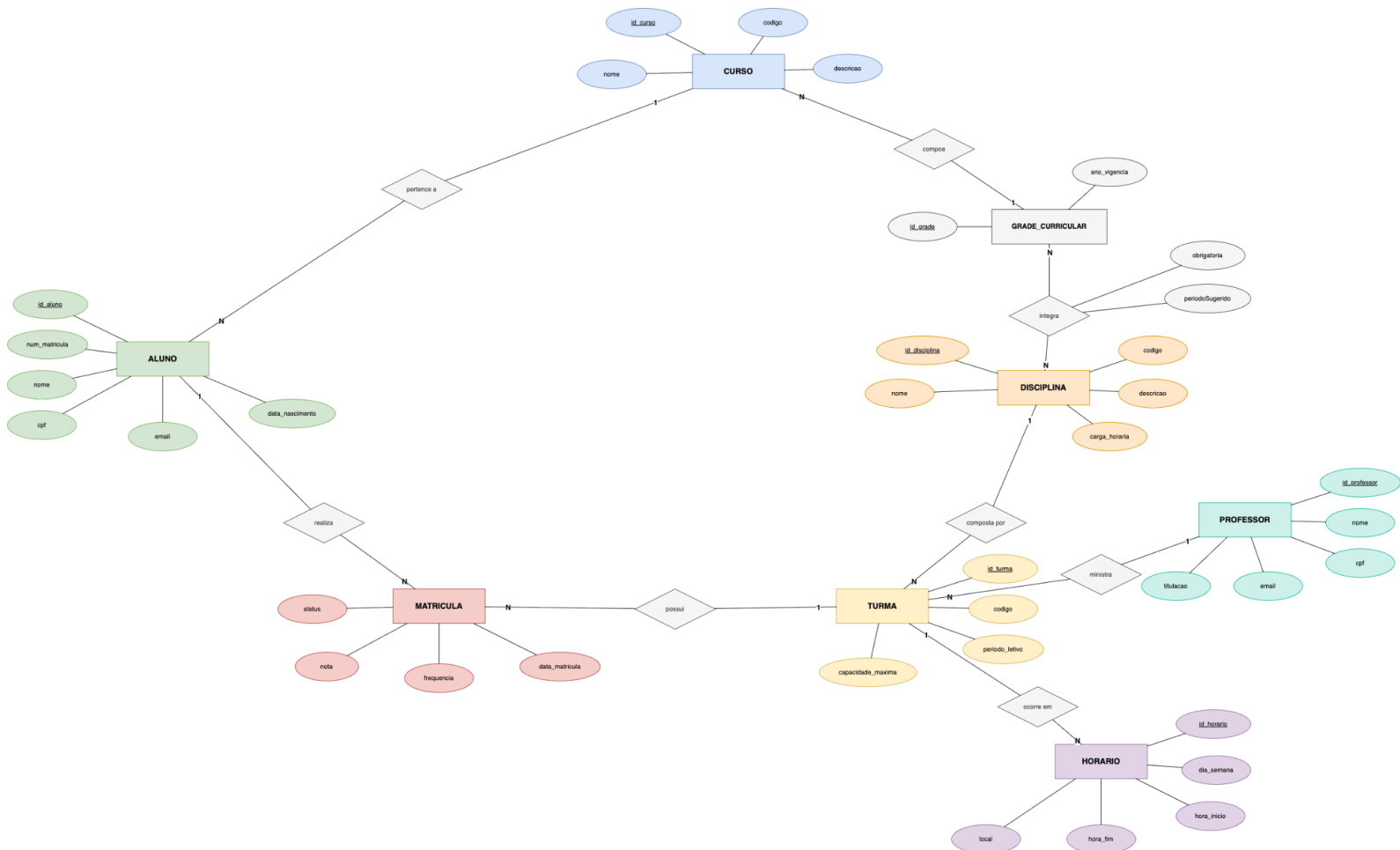
Para os alunos são armazenados dados como identificador, número de matrícula, nome e demais informações pessoais, além do vínculo com um curso. De cada curso devem ser armazenados identificador, código, nome e descrição.

Um curso pode possuir várias versões de grade curricular ao longo do tempo, identificadas pelo ano de vigência. Cada grade pertence a um único curso. As disciplinas são entidades independentes, com identificador, código, nome, descrição e carga horária, podendo integrar diferentes grades. A associação entre uma grade e uma disciplina registra se ela é obrigatória e o período sugerido para sua realização.

Para os professores são armazenados identificador, nome, CPF, e-mail e titulação. **As turmas vinculam uma disciplina a um professor**, armazenando também código, período letivo e capacidade máxima de alunos. **Cada turma pode ter vários horários associados**, cada qual com dia da semana, horário de início, horário de término e local.

É necessário **registrar a matrícula dos alunos nas turmas**. A matrícula representa o vínculo entre um aluno e uma turma, sendo responsável por armazenar também data de matrícula, status, nota e frequência. Um aluno pode estar matriculado em várias turmas, e uma turma pode possuir vários alunos matriculados. O sistema deve garantir que a capacidade máxima de cada turma seja respeitada e evitar a duplicidade de matrícula de um mesmo aluno em uma mesma turma.

3. DIAGRAMA ENTIDADE-RELACIONAMENTO (DER)



4. PROJETO RELACIONAL

curso(idCurso, codigo, nome, descricao)

professor(idProfessor, nome, cpf, email, titulacao)

disciplina(idDisciplina, codigo, nome, descricao, cargaHoraria)

grade_curricular(idGrade, idCurso, anoVigencia)

grade_disciplina(idGrade, idDisciplina, obrigatoria, periodoSugerido)

aluno(idAluno, numMatricula, nome, cpf, email, dataNascimento, idCurso)

turma(idTurma, codigo, periodoLetivo, capacidadeMaxima, idDisciplina, idProfessor)

horario(idHorario, diaSemana, horaInicio, horaFim, local, idTurma)

matricula(idAluno, idTurma, nota, frequencia, dataMatricula, status)

5. CRIAÇÃO DO BANCO DE DADOS E IMPLEMENTAÇÃO DE RESTRIÇÕES

```
CREATE DATABASE gestao_academica
```

```
CREATE TABLE curso (  
    idCurso      PRIMARY KEY,  
    codigo  VARCHAR(20)  NOT NULL UNIQUE,  
    nome    VARCHAR(100) NOT NULL,  
    descricao TEXT  
);
```

```
CREATE TABLE professor (  
    idProfessor   PRIMARY KEY,  
    nome    VARCHAR(150) NOT NULL,  
    cpf     CHAR(11)  NOT NULL UNIQUE,  
    email   VARCHAR(150) NOT NULL UNIQUE,  
    titulacao VARCHAR(30) NOT NULL  
    CHECK (titulacao IN ('Graduacao', 'Especializacao', 'Mestrado', 'Doutorado',  
    'Pos-Doutorado'))  
);
```

```
CREATE TABLE disciplina (  
    idDisciplina SERIAL PRIMARY KEY,  
    codigo  VARCHAR(20) NOT NULL UNIQUE,  
    nome    VARCHAR(150) NOT NULL,  
    descricao TEXT,  
    cargaHoraria SMALLINT NOT NULL  
    CHECK (cargaHoraria > 0 AND cargaHoraria % 15 = 0)  
);
```

```
CREATE TABLE grade_curricular (  
    idGrade   SERIAL PRIMARY KEY,  
    idCurso   INT NOT NULL REFERENCES curso(idCurso) ON DELETE  
    RESTRICT,  
    anoVigencia SMALLINT NOT NULL CHECK (anoVigencia >= 2000),  
    UNIQUE (idCurso, anoVigencia)  
);
```

```

CREATE TABLE grade_disciplina (
    idGrade    INT    NOT NULL REFERENCES grade_curricular(idGrade) ON
DELETE CASCADE,
    idDisciplina INT    NOT NULL REFERENCES disciplina(idDisciplina) ON
DELETE RESTRICT,
    obrigatoria BOOLEAN NOT NULL DEFAULT TRUE,
    periodoSugerido SMALLINT CHECK (periodoSugerido BETWEEN 1 AND
10),
    PRIMARY KEY (idGrade, idDisciplina)
);

```

```

CREATE TABLE aluno (
    idAluno    SERIAL    PRIMARY KEY,
    numMatricula VARCHAR(20) NOT NULL UNIQUE,
    nome       VARCHAR(150) NOT NULL,
    cpf        CHAR(11)   NOT NULL UNIQUE,
    email      VARCHAR(150) NOT NULL UNIQUE,
    dataNascimento DATE    NOT NULL
    CHECK (dataNascimento BETWEEN '1930-01-01' AND CURRENT_DATE -
INTERVAL '15 years'),
    idCurso    INT    NOT NULL REFERENCES curso(idCurso) ON DELETE
RESTRICT
);

```

```

CREATE TABLE turma (
    idTurma    SERIAL    PRIMARY KEY,
    codigo     VARCHAR(20) NOT NULL,
    periodoLetivo VARCHAR(10) NOT NULL
    capacidadeMaxima SMALLINT NOT NULL CHECK (capacidadeMaxima
>= 1),
    idDisciplina INT    NOT NULL REFERENCES disciplina(idDisciplina)
ON DELETE RESTRICT,
    idProfessor INT    NOT NULL REFERENCES professor(idProfessor)
ON DELETE RESTRICT,
    UNIQUE (codigo, periodoLetivo)
);

```

```

CREATE TABLE horario (
    idHorario SERIAL    PRIMARY KEY,

```

```

diaSemana VARCHAR(10) NOT NULL
    CHECK (diaSemana IN ('SEG', 'TER', 'QUA', 'QUI', 'SEX', 'SAB')),
horaInicio TIME NOT NULL,
horaFim TIME NOT NULL,
local VARCHAR(50) NOT NULL,
idTurma INT NOT NULL REFERENCES turma(idTurma) ON DELETE
CASCADE,
    CHECK (horaFim > horaInicio)
);

```

```

CREATE TABLE matricula (
    idAluno INT NOT NULL REFERENCES aluno(idAluno) ON DELETE
    RESTRICT,
    idTurma INT NOT NULL REFERENCES turma(idTurma) ON
    DELETE RESTRICT,
    dataMatricula DATE NOT NULL DEFAULT CURRENT_DATE,
    status VARCHAR(20) NOT NULL DEFAULT 'Ativa'
    CHECK (status IN ('Ativa', 'Trancada', 'Aprovado', 'Reprovado', 'Cancelada')),
    nota NUMERIC(4,2) CHECK (nota BETWEEN 0 AND 10),
    frequencia NUMERIC(5,2) CHECK (frequencia BETWEEN 0 AND 100),
    PRIMARY KEY (idAluno, idTurma)
);

```

Validação via Terminal

1) Validação DDL

```

SELECT table_name FROM information_schema.tables WHERE table_schema =
'public' ORDER BY table_name;
SELECT tc.table_name, tc.constraint_name, tc.constraint_type
FROM information_schema.table_constraints tc
WHERE tc.table_schema = 'public'
ORDER BY tc.table_name, tc.constraint_type;

```

terminal	output	debug console	playwright	problems
table_name	constraint_name	constraint_type		
aluno	aluno_idaluno_not_null	CHECK		
aluno	aluno_nummatricula_not_null	CHECK		
aluno	aluno_idcurso_not_null	CHECK		
aluno	aluno_datanascimento_not_null	CHECK		
aluno	aluno_email_not_null	CHECK		
aluno	aluno_cpf_not_null	CHECK		
aluno	aluno_nome_not_null	CHECK		
aluno	aluno_datanascimento_check	CHECK		
aluno	aluno_idcurso_fkey	FOREIGN KEY		
aluno	aluno_pkey	PRIMARY KEY		
aluno	aluno_nummatricula_key	UNIQUE		
aluno	aluno_email_key	UNIQUE		
aluno	aluno_cpf_key	UNIQUE		
curso	curso_idcurso_not_null	CHECK		
curso	curso_nome_not_null	CHECK		
curso	curso_codigo_not_null	CHECK		
curso	curso_pkey	PRIMARY KEY		
curso	curso_codigo_key	UNIQUE		
disciplina	disciplina_cargahoraria_not_null	CHECK		
disciplina	disciplina_codigo_not_null	CHECK		
disciplina	disciplina_nome_not_null	CHECK		
disciplina	disciplina_cargahoraria_check	CHECK		
disciplina	disciplina_iddisciplina_not_null	CHECK		
disciplina	disciplina_pkey	PRIMARY KEY		
disciplina	disciplina_codigo_key	UNIQUE		
grade_curricular	grade_curricular_idcurso_not_null	CHECK		
grade_curricular	grade_curricular_anovigencia_not_null	CHECK		
grade_curricular	grade_curricular_anovigencia_check	CHECK		
grade_curricular	grade_curricular_idgrade_not_null	CHECK		
grade_curricular	grade_curricular_idcurso_fkey	FOREIGN KEY		
grade_curricular	grade_curricular_pkey	PRIMARY KEY		
grade_curricular	grade_curricular_idcurso_anovigencia_key	UNIQUE		
grade_disciplina	grade_disciplina_idgrade_not_null	CHECK		
grade_disciplina	grade_disciplina_iddisciplina_not_null	CHECK		
grade_disciplina	grade_disciplina_obrigatoria_not_null	CHECK		

2) Validação FKs

```
SELECT kcu.table_name, kcu.column_name, ccu.table_name AS
tabela_referenciada
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu ON tc.constraint_name =
kcu.constraint_name
JOIN information_schema.constraint_column_usage ccu ON
tc.constraint_name = ccu.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY' ORDER BY kcu.table_name;
```

```

o → pibd psql -d gestao_academica
psql (18.3 (Homebrew))
Type "help" for help.

gestao_academica=# SELECT kcu.table_name, kcu.column_name, ccu.table_name AS tabela_referenciada
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu ON tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage ccu ON tc.constraint_name = ccu.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY' ORDER BY kcu.table_name;

```

table_name	column_name	tabela_referenciada
aluno	idcurso	curso
grade_curricular	idcurso	curso
grade_disciplina	idgrade	grade_curricular
grade_disciplina	iddisciplina	disciplina
horario	idturma	turma
matricula	idturma	turma
matricula	idaluno	aluno
turma	iddisciplina	disciplina
turma	idprofessor	professor

```

(9 rows)

gestao_academica=#

```

6. CRIAÇÃO DE ÍNDICES

```

CREATE INDEX idx_aluno_num_matricula ON aluno(numMatricula);
CREATE INDEX idx_aluno_curso ON aluno(idCurso);
CREATE INDEX idx_turma_periodo_letivo ON turma(periodoLetivo);
CREATE INDEX idx_turma_professor ON turma(idProfessor);
CREATE INDEX idx_matricula_aluno ON matricula(idAluno);
CREATE INDEX idx_matricula_status ON matricula(status);
CREATE INDEX idx_grade_disciplina_grade ON grade_disciplina(idGrade);
CREATE INDEX idx_horario_turma ON horario(idTurma);

```

Validação via Terminal

Query

```

SELECT
  tablename,
  indexname,
  indexdef
FROM pg_indexes
WHERE schemaname = 'public'
ORDER BY tablename, indexname;

```

Resultado

tablename	indexname	indexdef
aluno	aluno_cpf_key	CREATE UNIQUE INDEX aluno_cpf_key ON public.aluno USING btree (cpf)
aluno	aluno_email_key	CREATE UNIQUE INDEX aluno_email_key ON public.aluno USING btree (email)
aluno	aluno_nummatricula_key	CREATE UNIQUE INDEX aluno_nummatricula_key ON public.aluno USING btree (nummatricula)
aluno	aluno_pkey	CREATE UNIQUE INDEX aluno_pkey ON public.aluno USING btree (idaluno)
aluno	idx_aluno_curso	CREATE INDEX idx_aluno_curso ON public.aluno USING btree (idcurso)
aluno	idx_aluno_num_matricula	CREATE INDEX idx_aluno_num_matricula ON public.aluno USING btree (nummatricula)
curso	curso_codigo_key	CREATE UNIQUE INDEX curso_codigo_key ON public.curso USING btree (codigo)
curso	curso_pkey	CREATE UNIQUE INDEX curso_pkey ON public.curso USING btree (idcurso)
disciplina	disciplina_codigo_key	CREATE UNIQUE INDEX disciplina_codigo_key ON public.disciplina USING btree (codigo)
disciplina	disciplina_pkey	CREATE UNIQUE INDEX disciplina_pkey ON public.disciplina USING btree (iddisciplina)
grade_curricular	grade_curricular_idcurso_anovigencia_key	CREATE UNIQUE INDEX grade_curricular_idcurso_anovigencia_key ON public.grade_curricular USING btree (idcurso, anovigencia)
grade_curricular	grade_curricular_pkey	CREATE UNIQUE INDEX grade_curricular_pkey ON public.grade_curricular USING btree (idgrade)

7. INSERÇÃO DE DADOS DE TESTE

INSERT INTO curso (codigo, nome, descricao) VALUES

('BCC', 'Ciência da Computação', 'Bacharelado com ênfase em fundamentos teóricos e práticos da computação'),
('BSI', 'Sistemas de Informação', 'Bacharelado voltado à análise, projeto e gestão de sistemas de informação'),
('BEC', 'Engenharia de Computação', 'Bacharelado integrando hardware, software e sistemas embarcados'),
('BMA', 'Matemática', 'Bacharelado em matemática pura e aplicada com optativas em computação'),
('BFI', 'Física', 'Bacharelado em física teórica e experimental com laboratórios avançados'),
('BMB', 'Biomedicina', 'Bacharelado voltado ao diagnóstico laboratorial e à pesquisa biomédica'),
('BAD', 'Administração', 'Bacharelado em gestão organizacional, finanças e estratégia empresarial'),
('BDI', 'Direito', 'Bacharelado em ciências jurídicas com ênfase em direito digital'),
('BPS', 'Psicologia', 'Bacharelado e licenciatura em psicologia clínica e organizacional'),
('BBI', 'Biotecnologia', 'Bacharelado interdisciplinar em biologia molecular e engenharia genética');

INSERT INTO professor (nome, cpf, email, titulacao) VALUES

('Roberto Sperandio Bigonha', '12345678909', 'roberto.bigonha@dc.ufscar.br', 'Pos-Doutorado'),

('Elaine Parros Machado de Sousa', '98765432100', 'elaine.sousa@dc.ufscar.br',
'Doutorado'),
('Caetano Traina Junior', '11122233344', 'caetano.traina@dc.ufscar.br',
'Pos-Doutorado'),
('Agma Juci Machado Traina', '22233344455', 'agma.traina@dc.ufscar.br',
'Doutorado'),
('José Augusto Baranauskas', '33344455566', 'jose.baranauskas@dc.ufscar.br',
'Doutorado'),
('Maria Cristina Ferreira de Oliveira', '44455566677',
'mcristina.oliveira@dc.ufscar.br', 'Doutorado'),
('Alneu de Andrade Lopes', '55566677788', 'alneu.lopes@dc.ufscar.br',
'Doutorado'),
('Thiago Alexandre Salgueiro Pardo', '66677788899',
'thiago.pardo@dc.ufscar.br', 'Doutorado'),
('Diego Furtado Silva', '77788899900', 'diego.furtado@dc.ufscar.br',
'Mestrado'),
('Marcela Xavier Ribeiro', '88899900011', 'marcela.ribeiro@dc.ufscar.br',
'Doutorado'),
('Luciano Antonio Digiampietri', '99900011122',
'luciano.digiampietri@dc.ufscar.br', 'Doutorado'),
('Ricardo Marcondes Marcacini', '10011022033',
'ricardo.marcacini@dc.ufscar.br', 'Mestrado');

INSERT INTO disciplina (codigo, nome, descricao, cargaHoraria) VALUES

('SCC0101', 'Introdução à Computação', 'Fundamentos de lógica, algoritmos e linguagens de programação', 60),
('SCC0204', 'Algoritmos e Estruturas de Dados I', 'Estruturas lineares, recursão e análise de algoritmos básicos', 60),
('SCC0205', 'Algoritmos e Estruturas de Dados II', 'Árvores, grafos, hashing e algoritmos avançados de busca', 60),
('SCC0501', 'Organização de Computadores I', 'Representação de dados, circuitos combinacionais e sequenciais', 60),
('SCC0602', 'Banco de Dados I', 'Modelagem relacional, SQL, normalização e integridade de dados', 60),
('SCC0603', 'Banco de Dados II', 'Transações, concorrência, indexação e tópicos em NoSQL', 45),
('SCC0504', 'Sistemas Operacionais', 'Gerência de processos, memória, escalonamento e sistemas de arquivos', 60),
('SCC0506', 'Redes de Computadores', 'Modelos em camadas, protocolos TCP/IP, roteamento e segurança', 60),

('SCC0201', 'Programação Orientada a Objetos', 'Herança, polimorfismo, padrões de projeto e SOLID com Java', 60),
 ('SCC0715', 'Inteligência Artificial', 'Busca heurística, representação do conhecimento e aprendizado', 60),
 ('SME0101', 'Cálculo para Computação I', 'Limites, derivadas, integrais e aplicações em computação', 60),
 ('SME0201', 'Álgebra Linear e Aplicações', 'Vetores, matrizes, transformações lineares e autovalores', 60),
 ('SME0305', 'Probabilidade e Estatística', 'Distribuições de probabilidade, testes de hipótese e inferência', 45),
 ('SCC0901', 'Engenharia de Software I', 'Levantamento de requisitos, UML, processos ágeis e testes', 45),
 ('SCC0252', 'Desenvolvimento Web', 'HTML5, CSS3, JavaScript, REST APIs e frameworks modernos', 60);

INSERT INTO grade_curricular (idCurso, anoVigencia) VALUES

(1, 2020), (1, 2023),
 (2, 2019), (2, 2022),
 (3, 2021), (3, 2023),
 (4, 2020), (5, 2021),
 (6, 2022), (7, 2023);

INSERT INTO grade_disciplina (idGrade, idDisciplina, obrigatoria, periodoSugerido) VALUES

(1, 1, TRUE, 1), (1, 2, TRUE, 2), (1, 3, TRUE, 3), (1, 9, TRUE, 2), (1, 5, TRUE, 4),
 (2, 1, TRUE, 1), (2, 2, TRUE, 2), (2, 5, TRUE, 3), (2, 6, FALSE, 5), (2, 10, FALSE, 6),
 (3, 1, TRUE, 1), (3, 2, TRUE, 2), (3, 5, TRUE, 3), (3, 14, TRUE, 4),
 (4, 1, TRUE, 1), (4, 5, TRUE, 2), (4, 14, TRUE, 3), (4, 15, FALSE, 4),
 (5, 1, TRUE, 1), (5, 11, TRUE, 1);

INSERT INTO aluno (numMatricula, nome, cpf, email, dataNascimento, idCurso) VALUES

('758001', 'Ana Carolina Pereira Rodrigues', '35678901234',
 'ana.rodrigues@aluno.ufscar.br', '2001-03-15', 1),
 ('758002', 'Bruno Henrique dos Santos Lima', '46789012345',
 'bruno.lima@aluno.ufscar.br', '2000-07-22', 1),
 ('758003', 'Camila Vitória Ferreira Souza', '57890123456',
 'camila.souza@aluno.ufscar.br', '2001-11-30', 2),
 ('758004', 'Danilo Augusto Almeida Costa', '68901234567',
 'danilo.costa@aluno.ufscar.br', '2000-05-10', 2),

```

('759001', 'Estela Mariana Gomes Martins', '79012345678',
'estela.martins@aluno.ufscar.br', '2002-01-08', 1),
('759002', 'Felipe Correia Nascimento', '80123456789',
'felipe.nascimento@aluno.ufscar.br', '2001-09-17', 3),
('759003', 'Giovana Beatriz Ribeiro Carvalho', '91234567890',
'giovana.carvalho@aluno.ufscar.br', '2002-04-25', 3),
('760001', 'Heitor Lucas Melo Teixeira', '02345678901',
'heitor.teixeira@aluno.ufscar.br', '2003-06-12', 4),
('760002', 'Isabela Fontana Pires', '14456789012',
'isabela.pires@aluno.ufscar.br', '2002-08-03', 1),
('760003', 'João Pedro Moraes Cardoso', '25567890123',
'joao.cardoso@aluno.ufscar.br', '2003-02-19', 2),
('761001', 'Karina Luz Andrade Barbosa', '36678901234',
'karina.barbosa@aluno.ufscar.br', '2004-10-05', 5),
('761002', 'Leonardo Vieira Ramos', '47789012345',
'leonardo.ramos@aluno.ufscar.br', '2004-03-28', 4),
('761003', 'Mariana Cristina Nogueira Castro', '58890123456',
'mariana.castro@aluno.ufscar.br', '2003-07-14', 7),
('761004', 'Nicolas Eduardo Azevedo Fernandes', '69901234567',
'nicolas.fernandes@aluno.ufscar.br', '2004-12-01', 8),
('761005', 'Olivia Borges Machado', '71012345678',
'olivia.machado@aluno.ufscar.br', '2003-05-20', 1);

```

INSERT INTO turma (codigo, periodoLetivo, capacidadeMaxima, idDisciplina, idProfessor) VALUES

```

('SCC0101-A', '2025.1', 40, 1, 1),
('SCC0204-A', '2025.1', 35, 2, 2),
('SCC0602-A', '2025.1', 30, 5, 3),
('SCC0201-A', '2025.1', 40, 9, 4),
('SCC0504-A', '2025.1', 30, 7, 5),
('SCC0901-A', '2025.1', 35, 14, 6),
('SCC0252-A', '2025.1', 25, 15, 7),
('SCC0603-A', '2025.1', 20, 6, 8),
('SCC0715-A', '2025.1', 25, 10, 9),
('SME0101-A', '2025.1', 50, 11, 10),
('SCC0506-A', '2025.1', 30, 8, 11),
('SCC0205-A', '2025.1', 20, 3, 12);

```

INSERT INTO horario (diaSemana, horaInicio, horaFim, local, idTurma) VALUES

('Segunda', '08:00', '10:00', 'Sala AB1-04', 1),
 ('Quarta', '08:00', '10:00', 'Sala AB1-04', 1),
 ('Terca', '10:00', '12:00', 'Sala AB2-12', 2),
 ('Quinta', '10:00', '12:00', 'Sala AB2-12', 2),
 ('Segunda', '14:00', '16:00', 'Lab BD', 3),
 ('Quarta', '14:00', '16:00', 'Lab BD', 3),
 ('Terca', '08:00', '10:00', 'Sala AB1-08', 4),
 ('Quinta', '08:00', '10:00', 'Sala AB1-08', 4),
 ('Sexta', '08:00', '10:00', 'Sala AB2-06', 5),
 ('Segunda', '10:00', '12:00', 'Sala AB3-02', 6),
 ('Quarta', '10:00', '12:00', 'Sala AB3-02', 6),
 ('Terca', '14:00', '16:00', 'Lab Web', 7),
 ('Quinta', '14:00', '16:00', 'Lab Web', 7),
 ('Segunda', '16:00', '18:00', 'Sala AB2-14', 8),
 ('Sexta', '14:00', '16:00', 'Sala IA-01', 9);

INSERT INTO matricula (idAluno, idTurma, dataMatricula, status, nota, frequencia) VALUES

(1, 1, '2025-02-01', 'Aprovado', 8.5, 92.0),
 (1, 3, '2025-02-01', 'Aprovado', 7.2, 85.0),
 (1, 4, '2025-02-01', 'Ativa', NULL, 75.0),
 (2, 1, '2025-02-01', 'Reprovado', 4.0, 60.0),
 (2, 3, '2025-02-01', 'Aprovado', 9.0, 95.0),
 (3, 6, '2025-02-03', 'Ativa', NULL, 80.0),
 (3, 7, '2025-02-03', 'Ativa', NULL, 88.0),
 (4, 6, '2025-02-03', 'Trancada', NULL, 30.0),
 (5, 1, '2025-02-05', 'Ativa', NULL, 70.0),
 (5, 4, '2025-02-05', 'Ativa', NULL, 65.0),
 (6, 7, '2025-02-05', 'Aprovado', 9.5, 98.0),
 (7, 7, '2025-02-05', 'Ativa', NULL, 50.0),
 (8, 10, '2025-02-07', 'Ativa', NULL, 72.0),
 (9, 2, '2025-02-07', 'Ativa', NULL, 68.0),
 (10, 2, '2025-02-07', 'Aprovado', 7.8, 90.0),
 (11, 10, '2025-02-07', 'Ativa', NULL, 55.0),
 (12, 10, '2025-02-07', 'Reprovado', 3.5, 40.0),
 (15, 1, '2025-02-09', 'Ativa', NULL, 82.0);

Validação via Terminal

1) Quantidade de registros

Query

```
SELECT 'curso' AS tabela, COUNT(*) AS registros FROM curso
UNION ALL SELECT 'professor', COUNT(*) FROM professor
UNION ALL SELECT 'disciplina', COUNT(*) FROM disciplina
UNION ALL SELECT 'grade_curricular', COUNT(*) FROM grade_curricular
UNION ALL SELECT 'grade_disciplina', COUNT(*) FROM grade_disciplina
UNION ALL SELECT 'aluno', COUNT(*) FROM aluno
UNION ALL SELECT 'turma', COUNT(*) FROM turma
UNION ALL SELECT 'horario', COUNT(*) FROM horario
UNION ALL SELECT 'matricula', COUNT(*) FROM matricula
ORDER BY tabela;
```

Resultado

```
→ pibd psql -d gestao_academica
```

```
gestao_academica=# SELECT 'curso' AS tabela, COUNT(*) AS registros FROM curso
UNION ALL SELECT 'professor', COUNT(*) FROM professor
UNION ALL SELECT 'disciplina', COUNT(*) FROM disciplina
UNION ALL SELECT 'grade_curricular', COUNT(*) FROM grade_curricular
UNION ALL SELECT 'grade_disciplina', COUNT(*) FROM grade_disciplina
UNION ALL SELECT 'aluno', COUNT(*) FROM aluno
UNION ALL SELECT 'turma', COUNT(*) FROM turma
UNION ALL SELECT 'horario', COUNT(*) FROM horario
UNION ALL SELECT 'matricula', COUNT(*) FROM matricula
ORDER BY tabela;
```

tabela	registros
aluno	15
curso	10
disciplina	15
grade_curricular	10
grade_disciplina	20
horario	15
matricula	18
professor	12
turma	12

(9 rows)

2) Listagem base

Query

```
SELECT a.nome AS aluno, t.codigo AS turma, d.nome AS disciplina, m.nota,
m.frequencia, m.status
FROM matricula m
JOIN aluno a ON a.idAluno = m.idAluno
JOIN turma t ON t.idTurma = m.idTurma
JOIN disciplina d ON d.idDisciplina = t.idDisciplina
ORDER BY a.nome, t.codigo;
```

Resultado

aluno	turma	disciplina	nota	frequencia	status
Ana Carolina Pereira Rodrigues	SCC0101-A	Introdução à Computação	8.50	92.00	Aprovado
Ana Carolina Pereira Rodrigues	SCC0201-A	Programação Orientada a Objetos		75.00	Ativa
Ana Carolina Pereira Rodrigues	SCC0602-A	Banco de Dados I	7.20	85.00	Aprovado
Bruno Henrique dos Santos Lima	SCC0101-A	Introdução à Computação	4.00	60.00	Reprovado
Bruno Henrique dos Santos Lima	SCC0602-A	Banco de Dados I	9.00	95.00	Aprovado
Camila Vitória Ferreira Souza	SCC0252-A	Desenvolvimento Web		88.00	Ativa
Camila Vitória Ferreira Souza	SCC0901-A	Engenharia de Software I		80.00	Ativa
Daniilo Augusto Almeida Costa	SCC0901-A	Engenharia de Software I		30.00	Trancada
Estela Mariana Gomes Martins	SCC0101-A	Introdução à Computação		70.00	Ativa
Estela Mariana Gomes Martins	SCC0201-A	Programação Orientada a Objetos		65.00	Ativa
Felipe Correia Nascimento	SCC0252-A	Desenvolvimento Web	9.50	98.00	Aprovado
Giovana Beatriz Ribeiro Carvalho	SCC0252-A	Desenvolvimento Web		50.00	Ativa
Heitor Lucas Melo Teixeira	SME0101-A	Cálculo para Computação I		72.00	Ativa
Isabela Fontana Pires	SCC0204-A	Algoritmos e Estruturas de Dados I		68.00	Ativa
João Pedro Moraes Cardoso	SCC0204-A	Algoritmos e Estruturas de Dados I	7.80	90.00	Aprovado
Karina Luz Andrade Barbosa	SME0101-A	Cálculo para Computação I		55.00	Ativa
Leonardo Vieira Ramos	SME0101-A	Cálculo para Computação I	3.50	40.00	Reprovado
Olivia Borges Machado	SCC0101-A	Introdução à Computação		82.00	Ativa

(18 rows)

8. PROCEDURES, FUNCTIONS E TRIGGERS

a) FUNCTIONS

i) Função de Consulta de Vagas Disponíveis em Turma

```
CREATE OR REPLACE FUNCTION contar_vagas_disponiveis(p_idTurma INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
    v_capacidade INT;
    v_matriculados INT;
BEGIN
    SELECT capacidadeMaxima INTO v_capacidade
    FROM turma WHERE idTurma = p_idTurma;
```

```

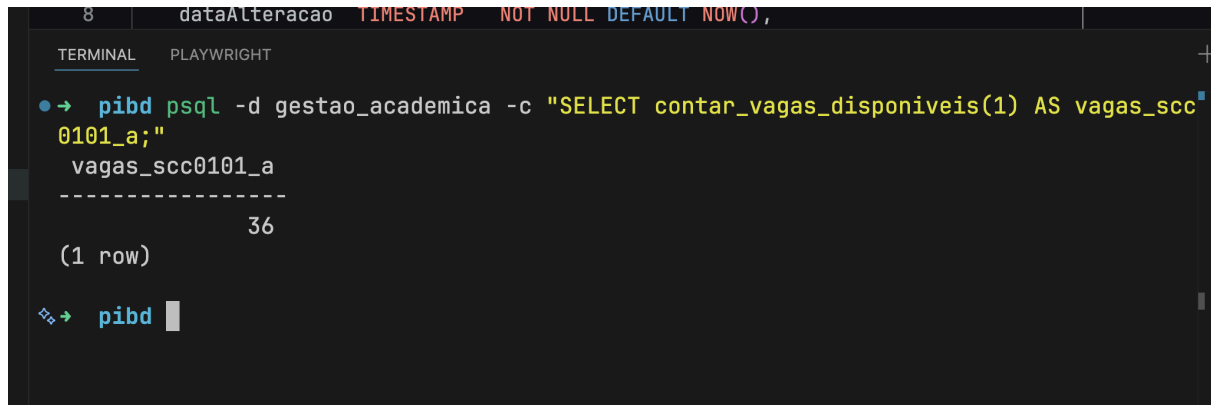
IF NOT FOUND THEN
    RAISE EXCEPTION 'Turma % não encontrada.', p_idTurma;
END IF;

SELECT COUNT(*) INTO v_matriculados
FROM matricula
WHERE idTurma = p_idTurma
AND status NOT IN ('Cancelada', 'Trancada');

RETURN v_capacidade - v_matriculados;
END;
$$;

```

Execução:



```

8 | dataAlteracao | TIMESTAMP | NOT NULL | DEFAULT NOW(), |
+-----+
| TERMINAL | PLAYWRIGHT |
+-----+
• → pibd psql -d gestao_academica -c "SELECT contar_vagas_disponiveis(1) AS vagas_scc0101_a;"
vagas_scc0101_a
-----
36
(1 row)

❖ → pibd

```

ii) Função de Cálculo do Índice de Rendimento Acadêmico (IRA)

```

CREATE OR REPLACE FUNCTION calcular_ira(p_idAluno INT)
RETURNS NUMERIC(4,2)
LANGUAGE plpgsql
AS $$
DECLARE
    v_ira NUMERIC(4,2);
BEGIN
    SELECT ROUND(
        SUM(m.nota * d.cargaHoraria) / NULLIF(SUM(d.cargaHoraria), 0),
        2
    ) INTO v_ira
    FROM matricula m
    JOIN turma t ON t.idTurma = m.idTurma
    JOIN disciplina d ON d.idDisciplina = t.idDisciplina
    WHERE m.idAluno = p_idAluno
    AND m.nota IS NOT NULL;

    RETURN COALESCE(v_ira, 0);
END;

```

\$\$;

Execução:

```
•→ pibd psql -d gestao_academica -c "SELECT a.nome, calcular_ira(a.idAluno) AS ira F
ROM aluno a WHERE
a.idAluno IN (1, 2, 10, 12) ORDER BY a.nome;"
      nome                | ira
-----+-----
Ana Carolina Pereira Rodrigues | 7.85
Bruno Henrique dos Santos Lima | 6.50
João Pedro Moraes Cardoso      | 7.80
Leonardo Vieira Ramos         | 3.50
(4 rows)

❖→ pibd
```

iii) Função de Verificação de Aprovação por Nota e Frequência

```
CREATE OR REPLACE FUNCTION aluno_aprovado(p_idAluno INT, p_idTurma INT)
RETURNS BOOLEAN
LANGUAGE plpgsql
AS $$
DECLARE
    v_nota NUMERIC(4,2);
    v_freq NUMERIC(5,2);
BEGIN
    SELECT nota, frequencia INTO v_nota, v_freq
    FROM matricula
    WHERE idAluno = p_idAluno AND idTurma = p_idTurma;

    IF NOT FOUND OR v_nota IS NULL OR v_freq IS NULL THEN
        RETURN NULL;
    END IF;

    RETURN v_nota >= 5.0 AND v_freq >= 75.0;
END;
$$;
```

Execução:

```
•→ pibd psql -d gestao_academica -c "SELECT aluno_aprovado(1, 1) AS ana_scc0101, alu
no_aprovado(2, 1)
AS bruno_scc0101;"
ana_scc0101 | bruno_scc0101
-----+-----
t           | f
(1 row)

❖→ pibd
```


b) PROCEDURES

i) Procedure de Realização de Matrícula com Validação de Vagas

```
CREATE OR REPLACE PROCEDURE realizar_matricula(p_idAluno INT, p_idTurma INT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_vagas INT;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM turma WHERE idTurma = p_idTurma) THEN
        RAISE EXCEPTION 'Turma % não encontrada.', p_idTurma;
    END IF;

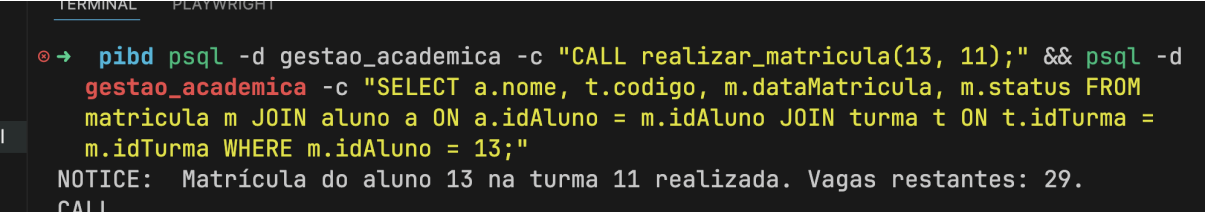
    IF EXISTS (SELECT 1 FROM matricula WHERE idAluno = p_idAluno AND idTurma =
p_idTurma) THEN
        RAISE EXCEPTION 'Aluno % já está matriculado na turma %.', p_idAluno, p_idTurma;
    END IF;

    v_vagas := contar_vagas_disponiveis(p_idTurma);
    IF v_vagas <= 0 THEN
        RAISE EXCEPTION 'Turma % sem vagas disponíveis.', p_idTurma;
    END IF;

    INSERT INTO matricula (idAluno, idTurma, dataMatricula, status)
    VALUES (p_idAluno, p_idTurma, CURRENT_DATE, 'Ativa');

    RAISE NOTICE 'Matrícula do aluno % na turma % realizada. Vagas restantes: %.', p_idAluno,
p_idTurma, v_vagas - 1;
END;
$$;
```

Execução:



```
TERMINAL PLAYWRIGHT
pibd psql -d gestao_academica -c "CALL realizar_matricula(13, 11);" && psql -d
gestao_academica -c "SELECT a.nome, t.codigo, m.dataMatricula, m.status FROM
matricula m JOIN aluno a ON a.idAluno = m.idAluno JOIN turma t ON t.idTurma =
m.idTurma WHERE m.idAluno = 13;"
NOTICE: Matrícula do aluno 13 na turma 11 realizada. Vagas restantes: 29.
CALL
```

ii) Procedure de Lançamento de Nota e Frequência

```
CREATE OR REPLACE PROCEDURE lancar_nota_frequencia(p_idAluno INT, p_idTurma INT,
p_nota NUMERIC(4,2), p_freq NUMERIC(5,2))
LANGUAGE plpgsql
```

```

AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM matricula
        WHERE idAluno = p_idAluno AND idTurma = p_idTurma AND status = 'Ativa'
    ) THEN
        RAISE EXCEPTION 'Matrícula ativa não encontrada para aluno % na turma %.', p_idAluno,
p_idTurma;
    END IF;

    UPDATE matricula
    SET nota = p_nota, frequencia = p_freq
    WHERE idAluno = p_idAluno AND idTurma = p_idTurma;

    RAISE NOTICE 'Nota %.2f e frequência %.1f%% lançadas para aluno % na turma %.', p_nota,
p_freq, p_idAluno, p_idTurma;
END;
$$;

```

Execução:

```

• → pibd psql -d gestao_academica -c "CALL lancar_nota_frequencia(1, 4, 6.5, 78.0);"
&&
psql -d gestao_academica -c "SELECT nota, frequencia, status FROM matricula WHERE
    idAluno = 1 AND idTurma = 4;"
NOTICE: Nota 6.5.2f e frequência 78.0.1f% lançadas para aluno 1 na turma 4.
CALL
  nota | frequencia | status
-----+-----+-----
  6.50 |      78.00 | Aprovado
(1 row)

```

iii) Procedure de Trancamento de Matrícula

```

CREATE OR REPLACE PROCEDURE trancar_matricula(p_idAluno INT, p_idTurma INT)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM matricula
        WHERE idAluno = p_idAluno AND idTurma = p_idTurma AND status = 'Ativa'
    ) THEN
        RAISE EXCEPTION 'Matrícula do aluno % na turma % não está ativa.', p_idAluno,
p_idTurma;
    END IF;

    UPDATE matricula SET status = 'Trancada'

```

```
WHERE idAluno = p_idAluno AND idTurma = p_idTurma;
```

```
RAISE NOTICE 'Matrícula do aluno % na turma % trancada com sucesso.', p_idAluno,  
p_idTurma;  
END;  
$$;
```

Execução:

```
⊙→ pibd psql -d gestao_academica -c "CALL trancar_matricula(3, 7);" && psql -d  
gestao_academica -c "SELECT status FROM matricula WHERE idAluno = 3 AND idTurma =  
7;"  
NOTICE: Matrícula do aluno 3 na turma 7 trancada com sucesso.  
CALL
```

c) TRIGGERS

i) Trigger de Auditoria de Alterações de Status de Matrícula

```
CREATE OR REPLACE FUNCTION fn_log_status_matricula()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
BEGIN  
IF OLD.status IS DISTINCT FROM NEW.status THEN  
INSERT INTO log_status_matricula (idAluno, idTurma, statusAnterior, statusNovo)  
VALUES (NEW.idAluno, NEW.idTurma, OLD.status, NEW.status);  
END IF;  
RETURN NEW;  
END;  
$$;
```

```
CREATE TRIGGER tg_log_status_matricula  
AFTER UPDATE ON matricula  
FOR EACH ROW  
EXECUTE FUNCTION fn_log_status_matricula();
```

Execução (mostra alterações que já foram executadas pelos comandos anteriores):

idlog	idaluno	idturma	statusanterior	statusnovo	dataalteracao	usuariobd
1	1	4	Ativa	Aprovado	2026-05-30 11:57:49.32076	mariacesa
2	3	7	Ativa	Trancada	2026-05-30 12:02:58.331386	mariacesa

(2 rows)

ii) Trigger de Atualização Automática de Status ao Lançar Nota

```

CREATE OR REPLACE FUNCTION fn_atualizar_status_nota()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.nota IS NOT NULL AND NEW.frequencia IS NOT NULL
    AND NEW.status NOT IN ('Cancelada', 'Trancada') THEN
        NEW.status := CASE
            WHEN NEW.nota >= 5.0 AND NEW.frequencia >= 75.0 THEN 'Aprovado'
            ELSE 'Reprovado'
        END;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER tg_atualizar_status_nota
BEFORE UPDATE OF nota, frequencia ON matricula
FOR EACH ROW
EXECUTE FUNCTION fn_atualizar_status_nota();

```

Execução (statusnovo):

idlog	idaluno	idturma	statusanterior	statusnovo	dataalteracao	usuariobd
1	1	4	Ativa	Aprovado	2026-05-30 11:57:49.32076	mariacesa
2	3	7	Ativa	Trancada	2026-05-30 12:02:58.331386	mariacesa

(2 rows)

iii) Trigger de Validação de Conflito de Sala e Horário

```

CREATE OR REPLACE FUNCTION fn_validar_horario_conflito()
RETURNS TRIGGER

```

```

LANGUAGE plpgsql
AS $$
DECLARE
    v_conflito INT;
BEGIN
    SELECT COUNT(*) INTO v_conflito
    FROM horario
    WHERE local = NEW.local
        AND diaSemana = NEW.diaSemana
        AND idHorario <> COALESCE(NEW.idHorario, -1)
        AND (NEW.horaInicio, NEW.horaFim) OVERLAPS (horaInicio, horaFim);

    IF v_conflito > 0 THEN
        RAISE EXCEPTION 'Conflito de horário: sala "%" já ocupada na % entre % e %.', NEW.local,
        NEW.diaSemana, NEW.horaInicio, NEW.horaFim;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER tg_validar_horario_conflito
BEFORE INSERT OR UPDATE ON horario
FOR EACH ROW
EXECUTE FUNCTION fn_validar_horario_conflito();

```

Execução:

```

...  → pibd psql -d gestao_academica -c "INSERT INTO horario (diaSemana, horaInicio, horaFim, local,
...      idTurma) VALUES ('Segunda', '09:00', '11:00', 'Sala AB1-04', 2);"
... ERROR:  Conflito de horário: sala "Sala AB1-04" já ocupada na Segunda entre 09:00:00 e 11:00:00.
... CONTEXT:  PL/pgSQL function fn_validar_horario_conflito() line 13 at RAISE
...  → pibd

```